



UNIDADE IV

Redes de Computadores
e Sistemas Distribuídos

Profa. Dra. Jacqueline Zonichenn

Conteúdo da Unidade IV

7. Sistemas distribuídos

7.1. Características dos sistemas distribuídos

7.2. Desafios dos sistemas distribuídos

7.3. Arquiteturas de sistemas distribuídos

7.4. Web services

8. Aplicações de sistemas distribuídos modernos

8.1. Microserviços

8.2. Blockchain

8.3. IoT

Contextualização

- A computação entre as décadas de 1940 e 1980 tinha um custo elevado.
- Computadores eram imensos e independentes (mainframes).

A partir dos anos 1980, dois avanços mudam esse contexto e proporcionam montar sistemas compostos por grandes quantidades de computadores, possibilitando os sistemas distribuídos:

- Microprocessadores;
- Redes de computadores.

Conceito de sistemas distribuídos

- “Um sistema distribuído é um conjunto de computadores independentes que se apresenta a seus usuários como um sistema único e coerente” (Tanenbaum, 2007, p.1).
- Por meio dos sistemas distribuídos, os componentes localizados em computadores que estão interligados em rede se comunicam e coordenam suas ações passando mensagens. Estes componentes podem ser de hardware (servidores) ou de software (aplicações e serviços).

Exemplos de sistemas distribuídos

- Internet: é o maior sistema distribuído, conectando globalmente usuários e serviços (e-mail, web, chat etc.) por meio de redes e protocolos que permitem a comunicação entre diferentes plataformas.
- Intranet: sistema distribuído privado de uma organização, composto por redes locais interligadas. Pode ter conexão com a internet, mas exige forte segurança para proteger dados corporativos sensíveis.

Exemplos de sistemas distribuídos

- Computação em nuvem: baseada em sistemas distribuídos, permite acesso remoto a recursos de computação e armazenamento por meio de provedores como AWS, Azure e Google Cloud, conectando múltiplos dispositivos.
- Aplicativos móveis: utilizam sistemas distribuídos ao se comunicarem com servidores remotos. Com redes sem fio e dispositivos portáteis, viabilizam a computação móvel com acesso de qualquer lugar.

Características dos sistemas distribuídos

- Uma das características principais dos sistemas distribuídos é que seus componentes estão dispersos geograficamente. Em um sistema distribuído, os elementos principais são chamados de nós ou nodes. Esses nós podem ser tanto máquinas físicas quanto processos virtuais executados em ambientes de nuvem.
- Cada nó de um sistema distribuído representa um processo de software que conhece outros processos e é capaz de se comunicar diretamente com eles. Essa comunicação acontece por meio do envio de mensagens, que servem para coordenar a execução das tarefas do sistema.

Características dos sistemas distribuídos

- O compartilhamento de recursos é outra importante característica dos sistemas distribuídos. Recursos podem ser compartilhados por diferentes servidores e consumidos por clientes. Serviços podem ser compartilhados por meio da internet, sendo encapsulados em objetos que podem ser acessados por outros objetos ou por aplicações do cliente.
- Outro fator marcante dos sistemas distribuídos é sua dinamicidade. Ou seja, os nós podem entrar ou sair do sistema a qualquer momento, sem comprometer o funcionamento geral ou a estrutura da rede.

Características dos sistemas distribuídos

Outras características dos sistemas distribuídos de acordo com Coulouris *et al.* (2013):

- Concorrência: sistemas distribuídos executam processos simultaneamente em diferentes máquinas que muitas vezes compartilham recursos. Isso gera disputas por esses recursos e exige mecanismos de coordenação para gerenciar a concorrência.
- Inexistência de relógio global: cada nó trabalha independentemente e tem sua própria noção de tempo. A coordenação depende da troca de mensagens e de mecanismos como controle de admissão e autenticação.
 - Falhas independentes: em sistemas distribuídos, falhas podem ocorrer em componentes, rede ou comunicação. Essas falhas nem sempre são detectadas de imediato ou percebidas pelos outros componentes.

Desafios dos sistemas distribuídos

- Comunicação entre os componentes: desafio de como garantir que todos os componentes possam se comunicar, considerando ambientes heterogêneos de rede, servidores, diferentes padrões de arquitetura e protocolos de comunicação.
- Escalabilidade: como fazer com que o sistema distribuído suporte o aumento no número de usuários ou requisições.
- Tolerância a falhas: como garantir a tolerância a falhas em um sistema com componentes distribuídos em servidores conectados a redes e ambientes remotos.

Desafios dos sistemas distribuídos

- Desempenho: como assegurar um desempenho aceitável em um sistema distribuído, em que os componentes estão interligados por redes que nem sempre possuem a mesma velocidade e no qual os componentes nem sempre têm a mesma capacidade de processamento, sem que surjam gargalos que comprometam o desempenho.
- Segurança: como certificar que somente os clientes válidos têm acesso aos serviços e componentes do sistema.

Comparação com sistemas centralizados

- Nos sistemas centralizados ou monolíticos, existe um único componente, que é o próprio sistema centralizado.
- Em sistemas distribuídos, é diferente, pois existem diversos componentes espalhados ou distribuídos por diferentes servidores, que podem ser heterogêneos, com diferentes sistemas operacionais, de diferentes fabricantes, com diferentes capacidades de processamento.

Interatividade

O suporte ao aumento inesperado no número de usuários ou requisições está relacionado à qual dos desafios dos sistemas distribuídos?

- a) Comunicação.
- b) Escalabilidade.
- c) Tolerância às falhas.
- d) Desempenho.
- e) Segurança.

Resposta

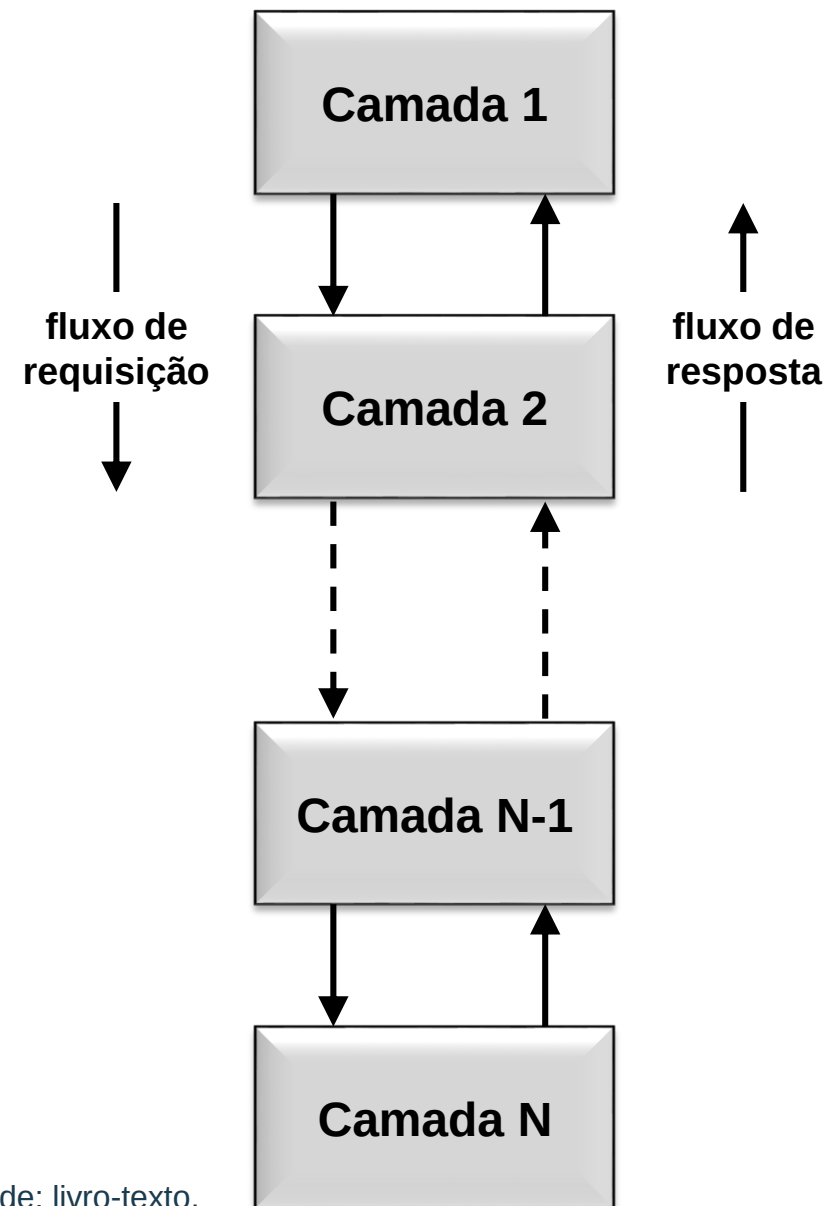
O suporte ao aumento inesperado no número de usuários ou requisições está relacionado à qual dos desafios dos sistemas distribuídos?

- a) Comunicação.
- b) Escalabilidade.**
- c) Tolerância às falhas.
- d) Desempenho.
- e) Segurança.

Arquiteturas de sistemas distribuídos

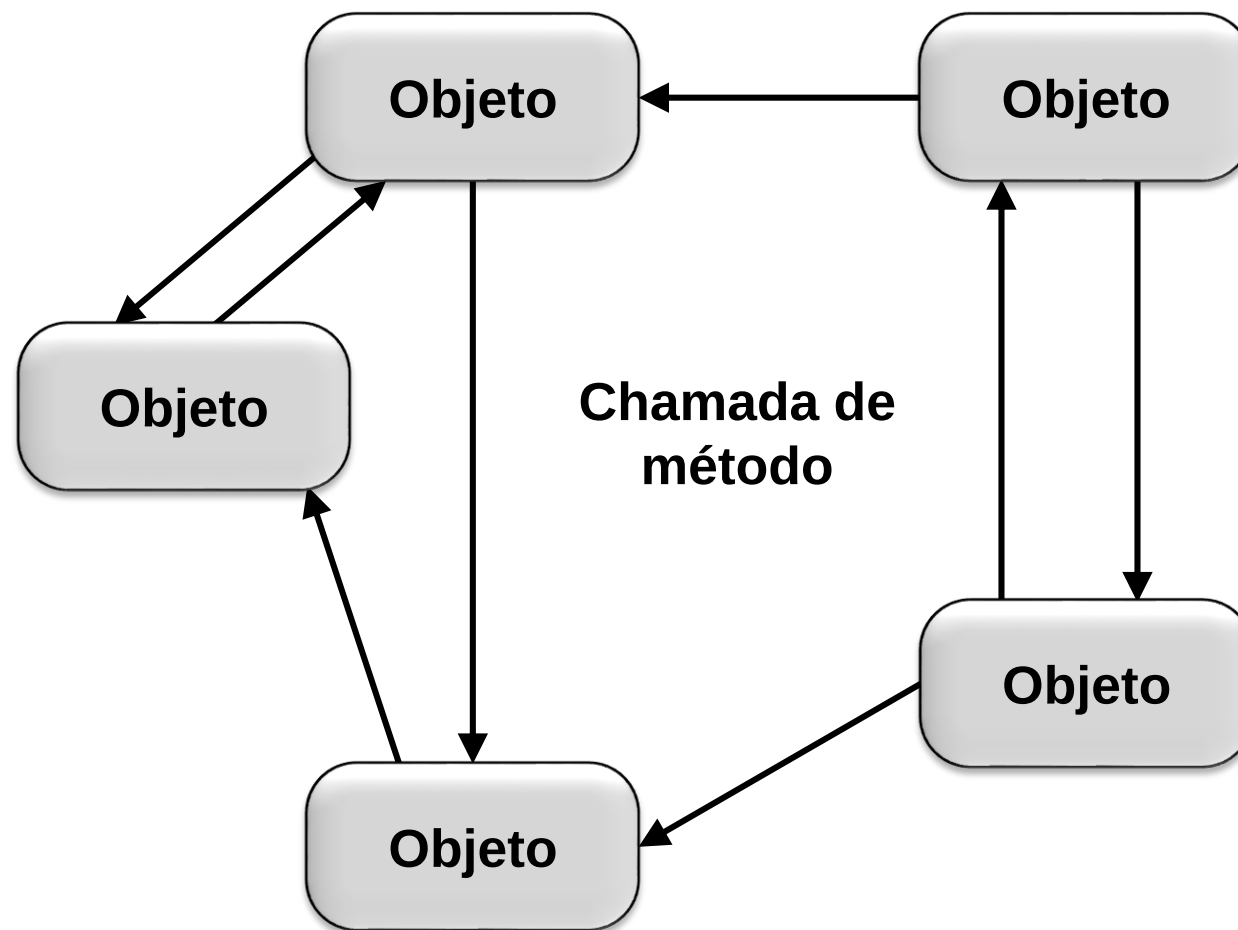
A seguir, exploraremos alguns dos estilos arquiteturais mais frequentemente utilizados no desenvolvimento de sistemas distribuídos, de acordo com Tanenbaum e Steen (2007):

- **Arquitetura em camadas:** organiza os componentes do sistema em níveis hierárquicos com funções definidas. Isso facilita a manutenção, a modularidade e a reutilização de código, sendo amplamente adotado em redes.



Arquiteturas de sistemas distribuídos

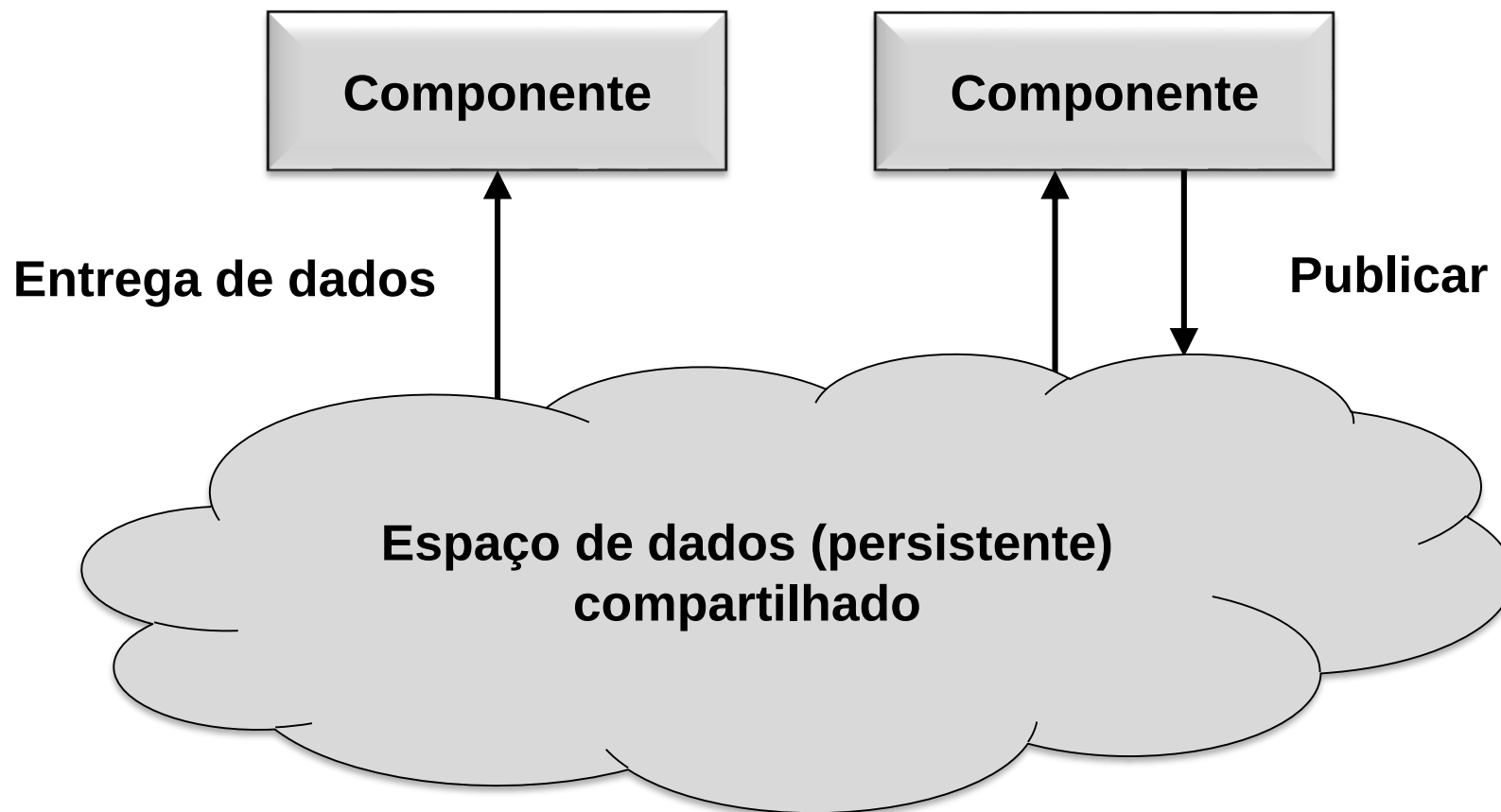
- Arquitetura baseada em objetos: trata os componentes como objetos autônomos que se comunicam via chamadas de procedimento remoto (RPC). Promove encapsulamento e reutilização, permitindo a criação de sistemas distribuídos compostos por serviços modulares.



Fonte: adaptado de: livro-texto.

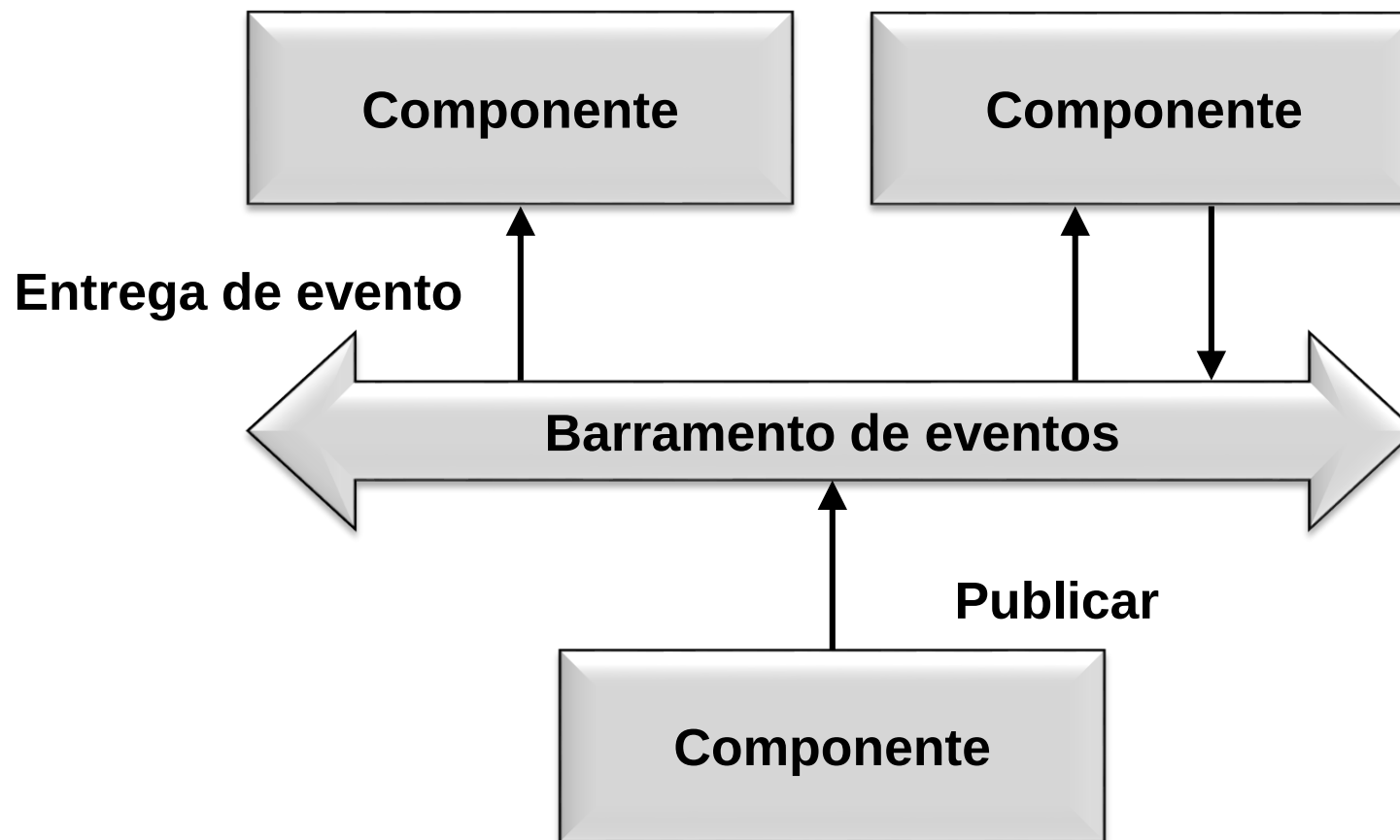
Arquiteturas de sistemas distribuídos

- Arquitetura centrada em dados: utiliza um repositório compartilhado para que os componentes do sistema leiam e gravem informações, promovendo interação indireta. A Web é um exemplo disso, com documentos acessados via HTTP. Tecnologias como o Amazon S3 seguem esse modelo para lidar com grandes volumes de dados.



Arquiteturas de sistemas distribuídos

- Arquitetura baseada em eventos: os componentes se comunicam por meio de eventos publicados em canais, sem dependência direta entre produtores e subscritores. Isso garante baixo acoplamento, permitindo maior escalabilidade e adaptabilidade do sistema, especialmente em ambientes dinâmicos.



Web Services

- Existem diversos níveis de abstração pertencentes à jornada de implementação de componentes com natureza distribuída.
- Um bloco de software em um sistema distribuído pode ser decomposto em partes menores. Essas partes são chamadas de web services, que permitem que diferentes aplicações e sistemas interajam entre si.

Web Services

- Um web service é uma função abstrata que descreve o que deve ser feito, como acessar dados, autenticar usuários ou processar pagamentos. Ele define funcionalidades acessíveis pela rede, geralmente pela internet; mas não é um componente físico, apenas uma descrição de serviços disponíveis.
- Já o agente concreto é a aplicação real que executa essas funções, como um servidor com código em Java ou Python. Esse agente processa as requisições dos clientes, enviando e recebendo mensagens para realizar o serviço descrito pelo web service.

Web Services

- Um determinado web service implementado pode usar um agente em um dia, escrito em uma linguagem de programação qualquer, e usar um agente diferente no dia seguinte, escrito em uma linguagem de programação diferente, mas exercendo a mesma funcionalidade.

As principais características dos web services vêm da seguinte tríade básica (W3C, 2004):

- descrição do serviço: para informar o que o serviço faz;
- descoberta do serviço: para os serviços serem contratados automaticamente;
- entrega do serviço: resultado efetivo da função ao cliente.

SOA – Service Oriented Architecture

- SOA (Service Oriented Architecture), ou Arquitetura Orientada a Serviços, é um conceito de arquitetura que busca disponibilizar as funcionalidades de um sistema como um serviço. Dessa forma, essas funcionalidades podem ser compartilhadas e reutilizadas entre aplicações. Seu principal objetivo é ser mais flexível em atender às necessidades do mercado.
- Nesse modelo, os web services se tornam os blocos de construção principais. Uma SOA baseada em web services se apoia em camadas padronizadas de tecnologia, com foco em expor funcionalidades da aplicação como serviços.

SOA – Service Oriented Architecture

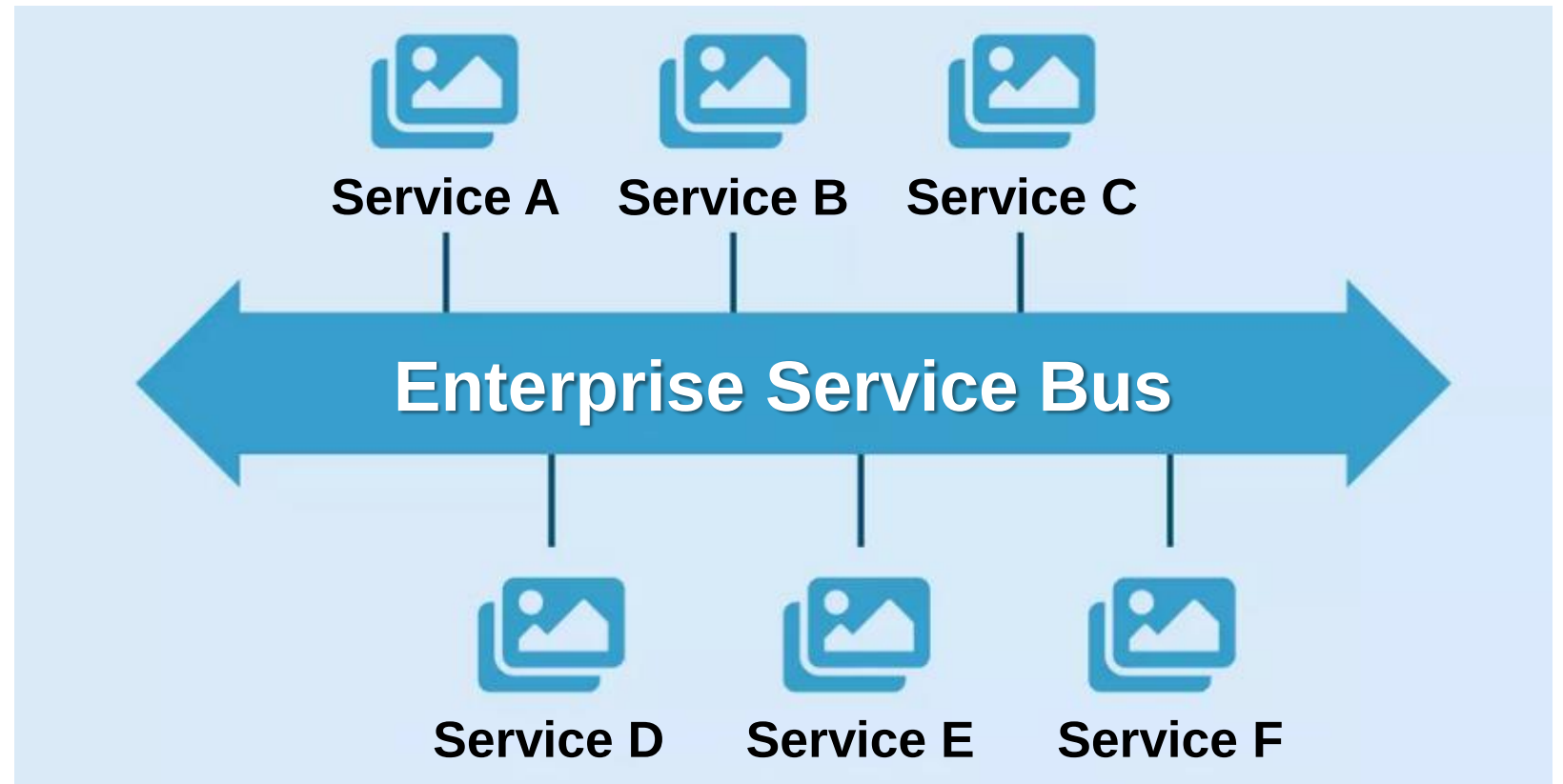
As principais vantagens da SOA são:

- alinhamento com as regras de negócio;
- diminuição do tempo de desenvolvimento;
- facilidade de manutenção devido ao baixo acoplamento entre as partes do sistema;
- flexibilidade durante mudanças, beneficiadas pelo isolamento da estrutura de um serviço;
- facilidade de agregar novas tecnologias a plataformas;
- possibilidade de reutilização de componentes.

ESB – Enterprise Service Bus

- Um dos componentes mais importantes da SOA é o ESB (Enterprise Service Bus), ou Barramento de Serviços Corporativos.
- O ESB é um modelo de arquitetura de software usado para projetar e implementar a comunicação entre aplicativos de software que interagem mutuamente em uma arquitetura SOA.

Fonte: livro-texto.



Virtualização

- A virtualização é uma técnica que possibilita o aproveitamento máximo da capacidade de uma máquina física, dividindo seus recursos de forma inteligente entre diferentes usuários ou ambientes. Assim, é possível dividir um único servidor físico em várias máquinas virtuais independentes, cada uma atuando como se fosse um servidor completo.
- A virtualização é uma tecnologia fundamental para sistemas distribuídos, pois permite criar ambientes escaláveis que facilitam o gerenciamento, a replicação e a alocação dinâmica de recursos entre múltiplos nós da rede.

Interatividade

Qual arquitetura utiliza um repositório compartilhado para que os componentes do sistema leiam e gravem informações, promovendo interação indireta entre eles?

- a) Arquitetura em camadas.
- b) Arquitetura hierárquica.
- c) Arquitetura baseada em objetos.
- d) Arquitetura centrada em dados.
- e) Arquitetura baseada em eventos.

Resposta

Qual arquitetura utiliza um repositório compartilhado para que os componentes do sistema leiam e gravem informações, promovendo interação indireta entre eles?

- a) Arquitetura em camadas.
- b) Arquitetura hierárquica.
- c) Arquitetura baseada em objetos.
- d) **Arquitetura centrada em dados.**
- e) Arquitetura baseada em eventos.

Microserviços

- Microserviços são uma forma específica de construir aplicações em que cada parte do sistema é desenvolvida como um serviço independente. Cada um desses serviços pode ser instalado, atualizado e ampliado de forma separada, sem depender diretamente da aplicação completa.

Microserviços

- Em sistemas distribuídos modernos, a arquitetura de microserviços quebra uma aplicação grande em vários serviços pequenos, independentes e especializados, que se comunicam entre si, normalmente via APIs. Cada microserviço é responsável por uma única função de negócio.
- Algumas peças são fundamentais em uma arquitetura moderna de sistemas distribuídos baseados em microserviços. Vejamos a seguir como cada um se conecta com esse modelo.

Containers

- Containers isolam tudo que uma aplicação precisa para funcionar: código, bibliotecas e configurações. Isso garante portabilidade e independência do ambiente.
- Containers também representam estruturas que armazenam objetos, como arrays e listas, ou ainda camadas como middleware que encapsulam funcionalidades.

Dockers

- Docker é a tecnologia que empacota e executa aplicações dentro de containers, tornando seu ambiente portátil e replicável.
- Antes do Docker, configurar sistemas distribuídos exigia processos manuais e demorados. Com ele, tudo pode ser automatizado e reutilizado com facilidade.

Kubernetes

- Kubernetes é uma plataforma de orquestração que gerencia e organiza múltiplos containers em produção.
- Ele automatiza tarefas como implantação, escalonamento e monitoramento, essencial quando há centenas ou milhares de containers interagindo em sistemas complexos.

API

- APIs são interfaces que permitem a comunicação entre softwares, especialmente entre microsserviços, seguindo padrões como REST.
- Elas definem como os serviços se conectam e expõem suas funcionalidades, isolando a lógica interna do sistema dos mecanismos de comunicação.

API Gateway

- O API Gateway centraliza o controle de acesso às APIs, funcionando como uma única porta de entrada e roteador de requisições.
- Além de organizar o tráfego, ele melhora a segurança, autentica usuários e controla o volume de acesso a cada serviço da arquitetura distribuída.

REST

- REST é um estilo arquitetural para comunicação entre sistemas usando o protocolo HTTP e identificadores como URLs.
- Ele usa métodos padronizados (GET, POST, PUT, DELETE) e troca dados normalmente em JSON, promovendo interoperabilidade e simplicidade na construção de APIs.

Conceitos relacionados

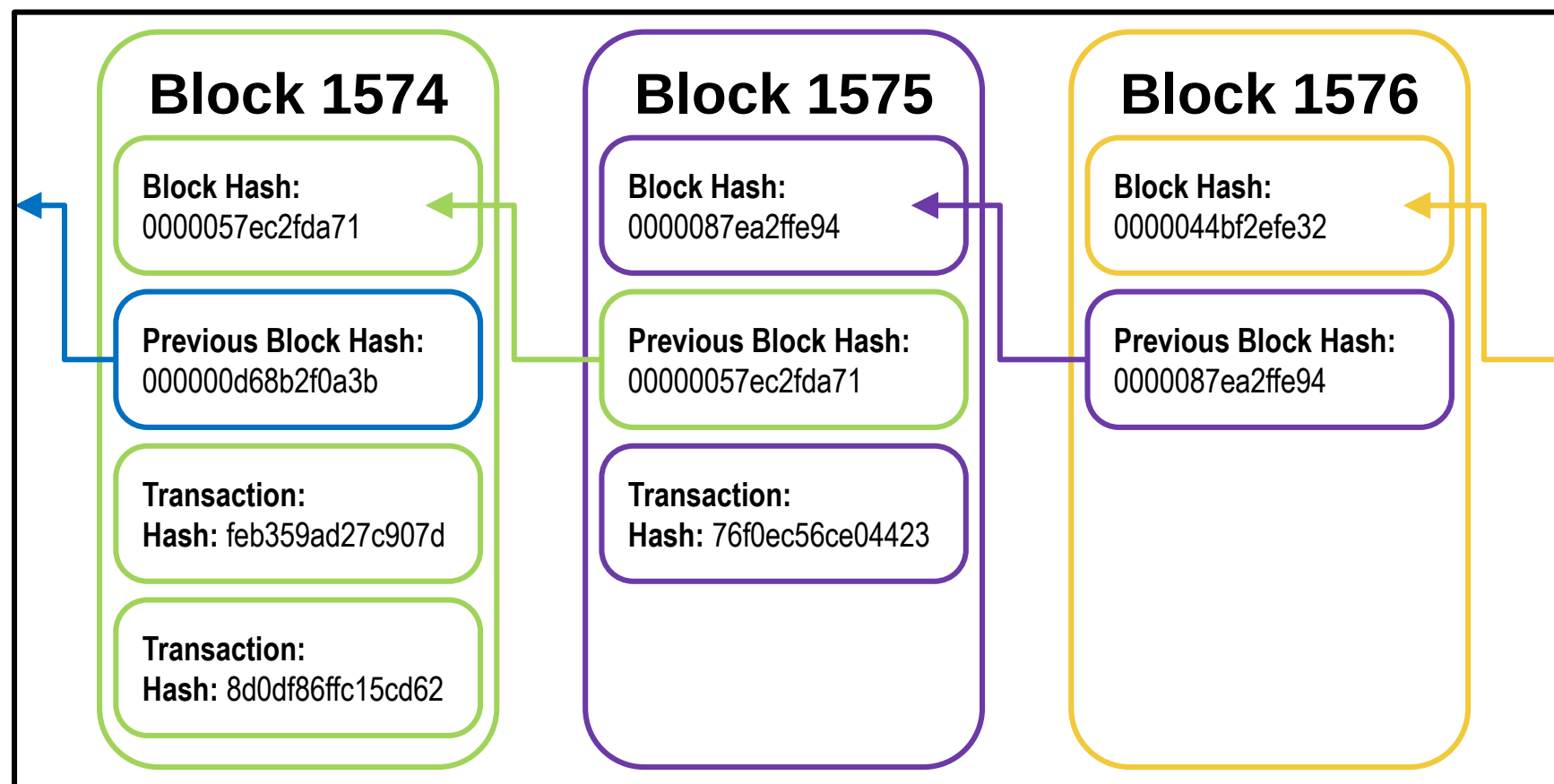
Conceito	Papel na arquitetura de sistemas distribuídos
Microserviços	Modelo arquitetural com serviços independentes e especializados.
APIs	Interface de comunicação entre os microserviços.
REST	Estilo arquitetural comumente usado para construir essas APIs.
Cointainers	Empacotam e isolam os microserviços.
Docker	Ferramenta para criar e executar containers.
Kubernetes	Orquestrador que gerencia e escala microserviços em containers.

Blockchain

- O Blockchain é um sistema distribuído para aplicação em cadeias de valor que exigem o registro de dados operacionais ou financeiros. Ele funciona como um livro-razão compartilhado em um sistema distribuído, que possui a função de registrar transações, executar contratos eletrônicos e também realizar o rastreamento de ativos.

Blockchain

- No blockchain, ocorre o armazenamento de blocos de dados vinculados por criptografia, que possui informações sobre uma aplicação específica. Cada bloco no blockchain contém todos os dados transacionais do sistema desde o seu início e a impressão digital única usada para confirmar as informações.



Interatividade

Que nome é dado às interfaces que permitem a comunicação entre softwares, especialmente entre microsserviços?

- a) APIs.
- b) REST.
- c) Containers.
- d) Dockers.
- e) Kubernetes.

Resposta

Que nome é dado às interfaces que permitem a comunicação entre softwares, especialmente entre microsserviços?

- a) APIs.
- b) REST.
- c) Containers.
- d) Dockers.
- e) Kubernetes.

Internet

Meio de comunicação caracterizado por:

- Canais genéricos e rede física distribuída, com rotas alternativas.

Protocolo TCP/IP, garante comunicação efetiva a partir de:

- Endereçamento IP;
 - Roteamento das mensagens;
 - Diferentes aplicações: Web, email, FTP etc.
-
- Motivação: rede entre universidades e entre governo e rede militar de defesa (Arpanet).
 - Comunicação voltada para pessoas.

Internet das Coisas

- Na IoT, objetos e dispositivos se comunicam entre si e com as aplicações em tempo real, por meio de sensores e atuadores.
- Por exemplo, um sensor capta a temperatura de determinado componente e transmite esse valor pela internet para uma aplicação.

Aplicações

- Previsão de demanda.
- Perfil de consumidor: projeto de produto.
- Manutenção preditiva.
- Indústria 4.0.
- Agricultura de precisão.
- Smart cities e mobilidade.
- Energia, etc.

Computação em nuvem

- A computação na nuvem é um modelo de serviço capaz de fornecer todo o tipo de processamento, infraestrutura e armazenamento de dados por meio da internet, tanto como componentes separados ou em uma plataforma completa.
- Permite ao usuário final acessar uma grande quantidade de aplicações e serviços em qualquer lugar, independentemente da plataforma, bastando para isso ter um terminal conectado à “nuvem”.

Características

Elasticidade e escalonamento

- Ilusão de recursos computacionais infinitos disponíveis para o uso.
- Capaz de fornecer rapidamente recursos em qualquer quantidade e a qualquer momento.

Self-service (auto-atendimento)

- Adquirir recursos computacionais de acordo com sua necessidade e de forma instantânea.
- Acesso em auto-atendimento.
- Solicitar, personalizar, pagar e usar os serviços desejados sem intervenção humana.

Características

Faturamento e medição por uso

- Usuário tem a opção de requisitar e utilizar somente a quantidade de recursos e serviços que ele julgar necessário
- As nuvens devem implementar recursos que garantam um eficiente comércio de serviços:
 - tarifação adequada;
 - faturamento;
 - monitoramento e otimização do uso.

Características

Amplo acesso à rede

- Recursos disponíveis na rede e acessados por meio de mecanismos padrões que permitam a utilização deles por plataformas heterogêneas.
 - Smartphones, laptops, PDAs.

Customização

- Grande disparidade entre as necessidades dos usuários.
 - Capacidade de personalização dos recursos da nuvem.

Vantagens

- Acesso aos dados e aplicações de qualquer lugar, desde que haja conexão de qualidade com a internet, trazendo assim mobilidade e flexibilidade aos usuários.
- Modelo de pagamento pelo uso (pay per use), em que o usuário paga somente o que necessita e evita desperdício de recursos.
- Escalabilidade, ampliando a disponibilidade de recursos conforme demanda.
- Riscos relacionados à infraestrutura minimizados, uma vez que o usuário não assume a responsabilidade pela infraestrutura.
- Facilidade de utilização dos serviços e compartilhamento de recursos.

Desafios

Segurança

- Armazenamento remoto (nuvem pública).
- Onde estão os dados?
- Quem acessa os dados? (privacidade).
- Como estão armazenados os dados? (integridade).
- Criptografia, controle de acesso, backup.

Interoperabilidade

- Portabilidade de aplicações + dados entre nuvens.
- Faltam padrões.

Desafios

Confiabilidade dos serviços (expectativa)

- As empresas que oferecem os serviços são avaliadas por sua reputação, principalmente pela capacidade de manter os dados seguros.

Disponibilidade

- Redundância na nuvem.
- Dependência da internet.
- Redundância entre nuvens.

Conclusão

Presença em diversas áreas:

- Doméstico, empresarial, comércio.
- Cotidiano: ferramentas, redes sociais, publicação e desenvolvimento de material.
- Transparente ao usuário (remoto ou local).
- Vantagens: financeira, flexibilidade e mobilidade.
- Desafios: segurança, padronização, modelo de negócio adequado.

Interatividade

Qual é o nome dado à interconexão de objetos e dispositivos com a internet para coleta e compartilhamento de dados?

- a) Blockchain.
- b) Internet Inteligente.
- c) Inteligência Artificial.
- d) Internet das Coisas.
- e) Computação em nuvem.

Resposta

Qual é o nome dado à interconexão de objetos e dispositivos com a internet para coleta e compartilhamento de dados?

- a) Blockchain.
- b) Internet Inteligente.
- c) Inteligência Artificial.
- d) Internet das Coisas.
- e) Computação em nuvem.

Referências

- COULOURIS, G. *et al. Sistemas distribuídos: conceitos e projeto*. São Paulo: Bookman Editora, 2013.
- TANENBAUM, A. S.; STEEN, M. *Sistemas distribuídos: princípios e paradigmas*. São Paulo: Pearson, 2007.

ATÉ A PRÓXIMA!